

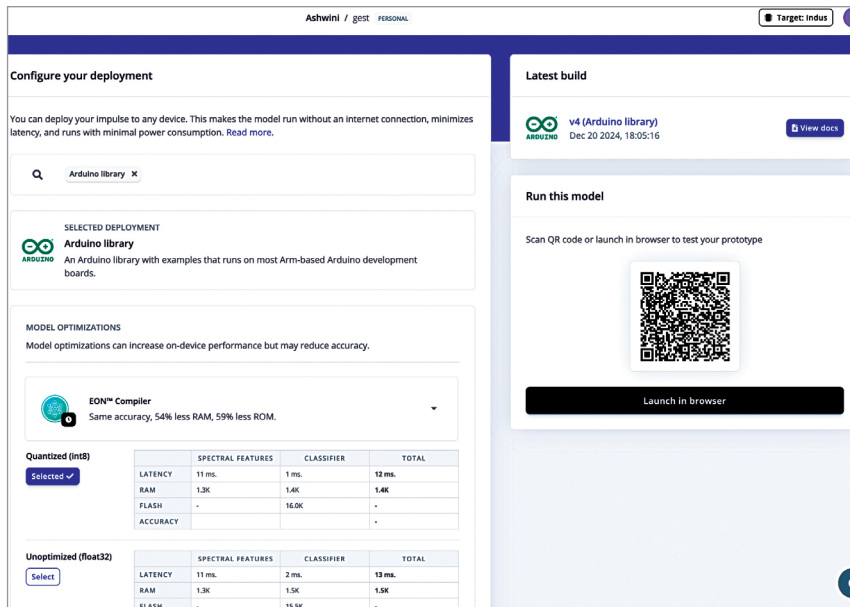


Fig. 9: Exporting the Arduino IDE library

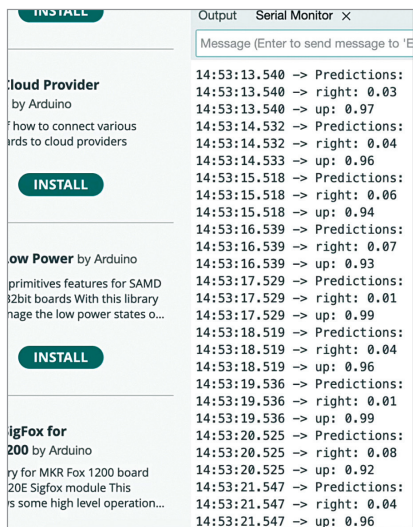


Fig. 10: AI predicting the gesture

main point where the graph changes due to motion and gestures. Next, move to the 'Classifier' section and select the neural layer, the number of training cycles, and the model type (SSD MobileNet or another). The default settings have been used, but they can be customised if preferred. Then, click 'Save and Train' to initiate the ML model training. After training, the model will display its learning progress in a graph. If

satisfied with the accuracy, proceed to test the model's performance. Fig. 7 shows the trained ML model.

Next, proceed to test the ML model. Subsequently, click on 'Classify All' to classify all the data and obtain the results. The model is considered decent if the accuracy is greater than 70%. However, if the results are unsatisfactory, increase the accuracy by adjusting the accuracy threshold. Alternatively, select the 'Retrain Model' option to tweak the training cycle and retrain the model. Fig. 8 shows the accuracy of the trained ML model.

Deployment

Go to the deployment section here. Select the MCU and chips for deployment. The IndusBoard, programmed in Arduino IDE, is used in this case. The Arduino library is selected. After that, select the 'Build' option, build the library, and automatically download the Arduino library to the laptop. If there is a need to tune and optimise the ML model, the EON tuner can be used; however, in this case, the model is used without tuning.

Next, open the Arduino IDE, go

DOWNLOAD SOURCE CODE & RESOURCES?



Scan this QR code OR else go to <https://sourcecode.electronicsforu.com> to access and download source code & other resources for this project

to 'Sketch' → 'Include Library' → 'Add Zip Library' and select the library just downloaded to add it. Next, go to the 'Library Examples' folder, then the 'ESP32' folder. Then, go to 'sensor inference' and open the example for the continuous sensor. The code can be modified to use the sensor and enable predictions based on the sensor data. The LSM303 sensor is used here, so the example code is modified accordingly. The library and code can be downloaded from *electronicsforu.com*.

Next, select the IndusBoard chip, specifically the ESP32-S2 Dev module, and upload the code. The compilation process takes over two minutes because the entire ML model is compiled and converted to binary format. Therefore, there will be a delay after uploading the code. Open the serial monitor and set the prediction for gesture performance. Perform the gesture; the device will detect it and display the results. Subsequently, create a function that executes specific actions when a particular gesture is detected. This function can then perform various tasks based on the gesture. Fig. 9 shows exporting the Arduino IDE library, while Fig. 10 shows the AI predicting the gesture. **EFY**

Bonus. You can watch the video of the tutorial of this DIY project at <https://www.electronicsforu.com/videos-slideshows/live-diy-running-ai-on-edge-using-indusboard-coin-and-mcus>

Ashwini Kumar Sinha, an IoT and AI enthusiast, is Tech Journalist at EFY.